Senior Fullstack Developer Technical Test

React, Node.js, MongoDB & AI Integration - Product and UI/UX Focus

TIME ALLOCATION	STACK	SUBMISSION
6-8 hours	TypeScript + MERN	GitHub + README + demo

Introduction

This assessment is for a senior fullstack engineer joining a distributed product team. It evaluates architecture, product judgement, maintainability, reliability, security, and attention to user experience. The solution should be polished and production-minded, without becoming unnecessarily large.

Project Task

Build an AI Interview Practice & Assessment App

A user uploads a resume, selects a target role, completes a short personalised mock interview, and receives a final assessment with scores, strengths, improvement areas, and recommended practice.

The AI layer may use a real provider or a clearly documented mock/fallback service. We are assessing the engineering design and user experience, not access to a paid AI account.

Core Requirements

1. Resume & Candidate Profile

- Upload a PDF or DOCX resume with file-size and type validation.
- Extract or simulate extraction of skills, experience, and a short candidate summary.
- Allow the user to review and edit the extracted profile before starting.

2. Interview Setup

- Select target role, seniority, and optionally paste a job description.
- Create five interview questions personalised to the resume and target role.
- Keep the AI provider behind an interface so it can be replaced or mocked.

3. Practice Session

- Show one question at a time with progress and clear navigation.
- Capture text answers; voice recording or speech-to-text is optional.
- Autosave answers and allow an interrupted session to resume after refresh or login.

4. Final Assessment

- Generate an overall score and category scores for relevance, clarity, structure, and communication.
- Show strengths, improvement areas, and one example of an improved answer.
- Persist the completed assessment and make it available from a simple history screen.

Backend Expectations

Use Node.js, Express, MongoDB, and TypeScript. A suggested API shape is below; equivalent designs are acceptable.

Endpoint	Purpose
POST /api/resumes	Upload and process a resume
POST /api/interviews	Create a personalised interview session
GET /api/interviews/:id	Load or resume an interview
POST /api/interviews/:id/answers	Save an answer safely and idempotently
POST /api/interviews/:id/complete	Complete the session and create assessment
GET /api/assessments/:id	Retrieve final assessment results

- Use clear domain boundaries for controllers, services, repositories, validation, and AI/file-processing adapters.
- Validate inputs and AI outputs, and provide deterministic fallback behaviour for provider failures.
- Address file security, rate limiting, sensitive-data handling, structured errors/logging, indexes, and authentication assumptions.

Frontend & UI/UX Expectations

- Create a clear multi-step flow: upload -> profile review -> interview -> assessment.
- Support responsive layouts, keyboard navigation, semantic HTML, accessible labels, and suitable contrast.
- Include thoughtful upload, loading, empty, validation, retry, network-error, autosave, and recovery states.
- Make results easy to scan with score cards, concise explanations, and actionable recommendations.

Senior-Level Engineering Expectations

Architecture: Explain boundaries, data flow, provider abstractions, and trade-offs.

Reliability: Handle duplicate requests, partial failures, session recovery, and fallback behaviour.

Security & Privacy: Treat resumes and answers as sensitive; validate files and protect secrets.

Testing & Operations: Test one critical backend flow and frontend interaction; include environment configuration and a health/monitoring note.

Evaluation Criteria

Area	Weight
Architecture, code quality, maintainability, and technical decisions	30%
Core product flow and data persistence	25%
Reliability, security, validation, and edge-case handling	20%
UI/UX quality, accessibility, and responsive design	15%
Tests, documentation, and developer experience	10%

Submission Requirements

1. GitHub repository with setup instructions, sample environment configuration, and seed/demo mode.
2. README with a small architecture diagram, key decisions, trade-offs, and future improvements.
3. Screenshots, a short demo video, or a deployed URL showing the complete user journey.